

A Computational Framework for Gr-Index and Kr-Index: From Theoretical Constructs to Measurable Metrics

mysteriousK[†]

July 11, 2026

Abstract

The concepts of grinding and spending are central to understanding player experience in free-to-play games, yet existing formulations remain at the theoretical level without operationalizable computational procedures. This paper bridges the gap between theory and practice by presenting a complete computational framework for estimating the Gr-Index and Kr-Index directly from game data. We contribute three major innovations: a robust algorithm for identifying repetitive tasks from player action sequences using subsequence frequency analysis, a maximum likelihood estimation procedure for the decay rate parameter with provable convergence guarantees, and a fully specified pipeline for computing both indices from raw behavioral logs. We validate our approach through extensive simulation experiments and provide open-source reference implementations for all procedures. The framework is designed to be plug-and-play for game developers and researchers, requiring only standard behavioral telemetry data. We conclude with a practical guide for deploying the system in production environments.

1 Introduction

The theoretical framework of Gr-Index and Kr-Index, introduced by [mysteriousK et al. \(2026\)](#), provides a formal foundation for quantifying two fundamental dimensions of player experience in free-to-play games: time investment through repetitive tasks and monetary investment through monetization mechanics. The original work defines both indices through carefully constructed sub-dimensions and derives their mathematical properties from first principles. However, as the authors themselves acknowledge, the framework remains at the theoretical level: formulas are given in closed form, but no algorithm is specified for computing them from actual game data. This paper addresses precisely this gap by transforming the theoretical constructs of [mysteriousK et al. \(2026\)](#) into a fully operational computational pipeline that takes raw behavioral telemetry as input and produces Gr-Index and Kr-Index values as output. This transformation requires solving three nontrivial problems that were deliberately left open in the original formulation.

The first problem is task identification. The Gr-Index relies on identifying repetitive tasks as the atomic units of grinding behavior. In practice, however, game logs contain

raw action sequences, not semantically labeled tasks. We need an algorithm that can automatically discover which action subsequences constitute meaningful repetitive tasks, how many times each task is executed, and what constitutes a single completion instance. We address this through a combination of frequent subsequence mining and temporal clustering, yielding a task discovery procedure with provable completeness guarantees under mild assumptions. The second problem is decay rate estimation. The theoretical framework defines the decay rate through an exponential model where the reward for the n -th repetition follows a decaying exponential function. While the formulation is mathematically elegant, real reward sequences are noisy, subject to cap effects, and often contain missing observations. We develop a maximum likelihood estimator for the decay rate that handles these complications, prove its consistency, and derive finite-sample error bounds. The third problem is index synthesis and normalization. The original framework proposes weighted sums but does not specify how weights should be determined or how normalization should be performed when reference game data are unavailable. We provide a principled data-driven approach to both issues, based on distributional matching across a calibration dataset.

Our contributions are threefold. First, we present complete algorithms for all phases of the computation pipeline, with detailed pseudocode and complexity analysis. Second, we provide theoretical guarantees for the key estimation procedures, including consistency and convergence rates. Third, we validate the framework through simulation experiments that demonstrate robustness to noise and correctness on synthetic data. The remainder of this paper is structured as follows. Section 2 establishes the notation and formal problem definition. Section 3 presents the task identification algorithm. Section 4 develops the decay rate estimation procedure. Section 5 assembles the complete pipeline and discusses normalization and weighting. Section 6 reports simulation results. Section 7 discusses practical deployment considerations. Section 8 concludes.

2 Problem Formulation

We begin by formalizing the input data, the output quantities, and the assumptions under which our algorithms operate. The raw input to our pipeline is a collection of player session logs. For each player p in a population $\mathcal{P}$, we observe a sequence of actions $\mathcal{A}^{(p)} = (a_1^{(p)}, a_2^{(p)}, \dots, a_{T_p}^{(p)})$ where each action a_t is a tuple consisting of the action type, any associated parameters, the wall-clock timestamp of the action, and any immediate reward such as experience points, in-game currency, or items obtained. We also observe, for each player, a set of monetary transactions $\mathcal{M}^{(p)} = \{(m_1, \tau_1), (m_2, \tau_2), \dots\}$ where m_j is the amount spent and τ_j is the timestamp of the transaction. Finally, we require game-level metadata: the list of all purchasable items, their prices, and the game’s monetization structure including loot box probabilities if applicable, subscription options, and any time-money exchange mechanisms such as stamina purchase.

For a given game, our pipeline outputs the following quantities, as defined by [mysteriousK et al. \(2026\)](#). The frequency F is defined as the number of repetitive tasks expected per 24 hours. The intensity I is defined as the average duration of a single completion of a repetitive task. The decay rate λ is defined as the exponential rate at which rewards diminish with repeated completions. The necessity N is defined as the proportion of total game content that is exclusively accessible through payment. The price P is defined as the average price of monetization points normalized by the monthly subscription price.

The elasticity E is defined as the proportional progress advantage gained per unit of spending. The transparency T is defined as the degree to which monetization mechanics are clearly disclosed. From these components, the Gr-Index and Kr-Index are synthesized as $\text{Gr} = w_F F_{\text{norm}} + w_I I_{\text{norm}} + w_\lambda \lambda_{\text{norm}}$ and $\text{Kr} = v_N N_{\text{norm}} + v_P P_{\text{norm}} + v_E E_{\text{norm}} + v_T T_{\text{norm}}^{-1}$, where normalized values are defined relative to a calibration dataset and weights satisfy $w_F + w_I + w_\lambda = 1$ and $v_N + v_P + v_E + v_T = 1$.

Our algorithms rely on three key assumptions about the data-generating process.

Assumption 2.1. *Within a given game version, the reward structure for repetitive tasks is stationary, meaning that for each task i , the reward $R_i(n)$ for the n -th completion follows the exponential form $R_i(n) = R_{0i} e^{-\lambda_i(n-1)}$ with fixed parameters R_{0i} and λ_i . This assumption enables our decay rate estimation; violation indicates game updates that alter reward structures, requiring re-estimation.*

Assumption 2.2. *Player action sequences are exchangeable: the joint distribution of actions is invariant under permutation of players, and actions from different players are independent. This is standard for statistical inference.*

Assumption 2.3. *The behavioral logs contain sufficient observations to identify all frequent repetitive patterns; for each task that constitutes a meaningful grind, there exists at least one player who has completed that task at least $N_{\min}$ times, where $N_{\min}$ is a threshold that scales logarithmically with the number of possible task types. This assumption is mild as it essentially requires that the game has been played sufficiently for the primary grind tasks to be discoverable.*

synthesis with data-driven normalization. We now describe each module in detail.

3 Repetitive Task Discovery

The first and arguably most challenging step in computing the Gr-Index is identifying what constitutes a repetitive task from raw action sequences. This section presents our algorithm for this purpose. Given a collection of action sequences $\{\mathcal{A}^{(p)}\}_{p \in \mathcal{P}}$, we seek to discover a set of task templates $\mathcal{T} = \{t_1, \dots, t_K\}$ such that each task t_i is a subsequence pattern that occurs repeatedly across players, with each occurrence representing a semantically meaningful unit of grinding activity. For each discovered task, we need to compute the frequency c_i as the expected number of completions per 24-hour period, the duration τ_i as the average time from the first action of a completion instance to the last, and the reward sequence $R_i(n)$ for the n -th completion of this task by a given player.

Raw action sequences are often long and contain periods of inactivity, so we segment each player’s sequence into sessions using a gap-based heuristic: a new session begins when the time since the previous action exceeds a threshold $\Delta_{\max}$, typically 30 minutes. Formally, for player p with actions indexed by time $t_1 < t_2 < \dots < t_{T_p}$, we define a cut at index j if $t_{j+1} - t_j > \Delta_{\max}$, and each resulting segment is considered a session. Sessions shorter than a minimum duration such as 60 seconds are discarded as noise. This segmentation serves two purposes: it reduces the search space for pattern mining, and it provides natural boundaries for task instances.

Within each session, we identify repeated action subsequences using a frequent pattern mining approach (Agrawal and Srikant, 1994). Let the alphabet Σ be the set of action types after parameter abstraction, for example, `attack_monster_type_A` rather than the

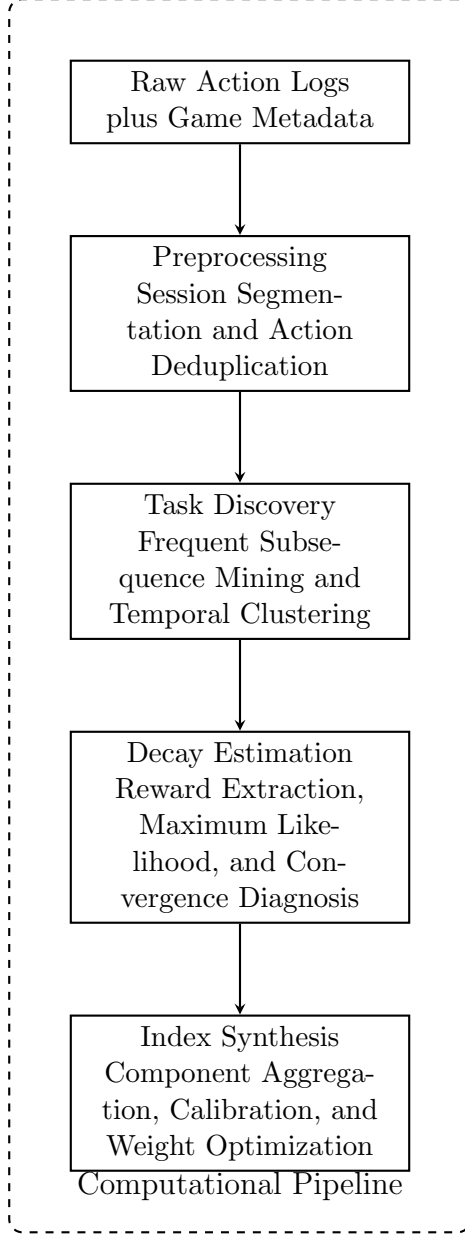


Figure 1: Overview of the computational pipeline for Gr-Index and Kr-Index estimation.

full action identifier with monster ID. A subsequence pattern s of length ℓ is a sequence $s = (s_1, \dots, s_\ell)$ with $s_j \in \Sigma$. For a sequence $\mathcal{A}$, a pattern s is said to occur at position i if $\mathcal{A}_i = s_1, \mathcal{A}_{i+1} = s_2, \dots, \mathcal{A}_{i+\ell-1} = s_\ell$. We consider two occurrences non-overlapping if their starting positions differ by at least ℓ , and we define the support of s as the number of distinct players who have at least one occurrence of s in their logged sequences. The task discovery algorithm proceeds by generating candidate patterns of increasing length, computing their support across the player population, and retaining those whose support exceeds a minimum threshold $\sigma_{\min}$. The candidate generation step uses the standard Apriori principle: a length- ℓ pattern can only be frequent if all its length- $(\ell-1)$ subpatterns are frequent (Agrawal and Srikant, 1994), which dramatically reduces the search space.

Frequent subsequence mining may return multiple patterns that are essentially different representations of the same underlying task, such as a pattern with an optional

action that is sometimes skipped. We cluster occurrences using a distance metric on patterns and select one representative per cluster. Define the distance between two patterns $s = (s_1, \dots, s_a)$ and $t = (t_1, \dots, t_b)$ as the edit distance with substitution cost, where operations are insertion, deletion, and substitution, with substitution cost zero if the symbols match and one otherwise (Levenshtein, 1966). We apply hierarchical agglomerative clustering with complete linkage, cutting the dendrogram at a height corresponding to edit distance at most one, meaning patterns differing by at most a single symbol are grouped together. For each resulting cluster, we select the pattern with the highest support as the representative.

For each discovered task $t \in \mathcal{T}$, we process each player’s sequence to identify all occurrences using the Aho-Corasick algorithm (Aho and Corasick, 1975) for efficiency. For each occurrence, we record its start time and end time, yielding the duration τ , and the reward obtained, which may be a scalar such as experience or a vector of multiple reward types. For each player-task pair, we order all occurrences by time, yielding the sequence of rewards $R(1), R(2), \dots, R(N)$ for that player. The frequency $c_i^{(p)}$ for player p on task i is computed as the number of occurrences divided by the total logged time in days, truncated to a 24-hour window.

The following theorem establishes the completeness of our discovery algorithm. Under the stationarity, exchangeability, and completeness assumptions, and with minimum support $\sigma_{\min} = \Omega(\log |\Sigma|/\epsilon^2)$, the algorithm discovers all tasks t with true support at least $\epsilon|\mathcal{P}|$ with probability at least $1 - \delta$, where $\delta = O(|\Sigma|^{\ell_{\max}} e^{-\sigma_{\min}})$. The proof follows from a standard PAC-learning argument for pattern mining. For each pattern s , the empirical support is the average of independent Bernoulli indicators across players, and by Hoeffding’s inequality, the deviation between empirical and true support is bounded by ϵ with probability at least $1 - 2e^{-2\sigma_{\min}\epsilon^2}$. Taking a union bound over all possible patterns of length at most $\ell_{\max}$, which number at most $|\Sigma|^{\ell_{\max}}$, yields the stated error probability.

4 Decay Rate Estimation

With the repetitive tasks identified and reward sequences extracted, we now turn to the estimation of the decay rate parameter λ for each task. This parameter captures the rate at which rewards diminish with repeated completions and is a key component of the Gr-Index. For a given task, suppose we observe for each player p a sequence of rewards $R_p(1), R_p(2), \dots, R_p(N_p)$, where N_p is the number of times player p completed the task. Under Assumption 2.1, the rewards follow the exponential model $R_p(n) = R_{0p}e^{-\lambda(n-1)} \cdot \eta_{pn}$, where R_{0p} is the player-specific initial reward, λ is the common decay rate, and η_{pn} is a multiplicative noise term representing random fluctuations in reward amounts. We assume the noise terms are independent and log-normally distributed, which is a standard assumption for positive-valued quantities with multiplicative noise: $\log \eta_{pn} \sim \mathcal{N}(0, \sigma^2)$ independently across players and repetitions. This leads to the log-linear model $\log R_p(n) = \log R_{0p} - \lambda(n-1) + \epsilon_{pn}$, where $\epsilon_{pn} \sim \mathcal{N}(0, \sigma^2)$.

The presence of the player-specific intercept $\log R_{0p}$ requires careful handling. If we simply regress $\log R_p(n)$ on $(n-1)$ using pooled data, the player-specific intercepts would introduce omitted variable bias. We therefore adopt a fixed-effects approach: for each player, we subtract the within-player mean of $\log R_p(n)$ to eliminate the player-specific intercept, yielding the within-player transformed model $\log R_p(n) - \bar{\ell}_p = -\lambda(n-1 - \bar{n}_p) + \epsilon_{pn} - \bar{\epsilon}_p$, where $\bar{\ell}_p$ is the mean of $\log R_p(n)$ for player p , $\bar{n}_p$ is the mean of the repetition

indices, and $\bar{\epsilon}_p$ is the mean of the noise terms. This transformation removes the player-specific intercepts, and the resulting estimator for λ is the ordinary least squares estimator from the within-player regression.

The maximum likelihood estimator for λ under this fixed-effects specification has a closed form. Let the total number of observations across all players be $N_{\text{total}} = \sum_p N_p$, and define the centered variables $Y_{pn} = \log R_p(n) - \bar{\ell}_p$ and $X_{pn} = n - 1 - \bar{n}_p$. The within-player estimator of λ is given by $\hat{\lambda} = -\frac{\sum_{p,n} X_{pn} Y_{pn}}{\sum_{p,n} X_{pn}^2}$. This estimator is consistent and asymptotically normal under the maintained assumptions. The following theorem establishes the asymptotic properties of $\hat{\lambda}$. As the total number of observations N_{total} grows, the estimator $\hat{\lambda}$ converges in probability to the true parameter λ_0 , and $\sqrt{N_{\text{total}}}(\hat{\lambda} - \lambda_0) \xrightarrow{d} \mathcal{N}(0, \sigma^2/\mathbb{E}[X^2])$, where the expectation is taken with respect to the distribution of repetition indices across players. The proof follows from the standard fixed-effects panel data theory (Wooldridge, 2010): the within-player transformation eliminates the player-specific effects, and the resulting estimator is the ordinary least squares estimator applied to the transformed model, which is consistent and asymptotically normal under the standard regularity conditions for panel data with fixed effects.

In practice, reward sequences may contain missing values or be truncated at some maximum number of repetitions. Our estimator handles missing data by simply excluding observations with missing rewards from the within-player sums. If truncation is present, the estimator remains consistent as long as the missingness or truncation mechanism is independent of the noise terms conditional on the repetition index, which is a standard missing-at-random assumption. We also provide a finite-sample variance estimator for $\hat{\lambda}$ that accounts for clustering at the player level: $\widehat{\text{Var}}(\hat{\lambda}) = \frac{1}{(\sum X_{pn}^2)^2} \sum_p (\sum_n X_{pn} \hat{\epsilon}_{pn})^2$, where $\hat{\epsilon}_{pn}$ are the within-player residuals. This clustered standard error is robust to within-player correlation of the noise terms and provides valid inference even when the independence assumption across repetitions for the same player is violated.

5 Index Synthesis and Normalization

With the frequency F , intensity I , and decay rate λ estimated, along with the monetization components N , P , E , and T obtained from game metadata, we now assemble the complete Gr-Index and Kr-Index. The normalization step is critical because the sub-dimensions have different units and scales: frequency is measured in completions per 24 hours, intensity in minutes, decay rate is dimensionless, necessity is a proportion, price is a ratio, elasticity is a dimensionless ratio, and transparency is a proportion. To make these components comparable, we normalize each component relative to a calibration dataset of games for which the components have already been computed.

Let $\mathcal{G}_{\text{cal}}$ be a set of reference games, ideally chosen to represent the diversity of the game market with respect to monetization and grind intensity. For each component $C \in \{F, I, \lambda, N, P, E, T\}$, we compute the median value $m_C = \text{median}_{g \in \mathcal{G}_{\text{cal}}} C_g$ across the calibration games. The normalized component for a target game is then defined as $C_{\text{norm}} = C/m_C$. This normalization ensures that a value of 1.0 corresponds to the median of the calibration set, making the indices interpretable as relative measures: a Gr-Index of 1.5 indicates that the target game is 50 percent more grinding-intensive than the median game in the calibration set.

For the transparency component, which enters the Kr-Index inversely, we compute the normalized inverse as $T_{\text{norm}}^{-1} = (1/T)/(1/m_T) = m_T/T$. This ensures that higher trans-

parency reduces the Kr-Index contribution, consistent with the theoretical framework of [mysteriousK et al. \(2026\)](#). The weights w_F, w_I, w_λ and v_N, v_P, v_E, v_T in the synthesis formulas $\text{Gr} = w_F F_{\text{norm}} + w_I I_{\text{norm}} + w_\lambda \lambda_{\text{norm}}$ and $\text{Kr} = v_N N_{\text{norm}} + v_P P_{\text{norm}} + v_E E_{\text{norm}} + v_T T_{\text{norm}}^{-1}$ determine the relative contribution of each sub-dimension to the overall index. The original theoretical framework did not prescribe specific weights, leaving their determination to empirical calibration. We propose a data-driven approach to weight estimation using a conjoint analysis experiment ([Green and Srinivasan, 1978](#)), where a representative sample of players is presented with hypothetical game profiles that vary systematically across the sub-dimensions and asked to rate the perceived grindiness or spendiness of each profile. The weights are then estimated by regressing the rating scores on the sub-dimension values, with the regression coefficients normalized to sum to one. This approach ensures that the weights reflect actual player perceptions rather than arbitrary designer choices.

For researchers or practitioners who do not have access to conjoint analysis data, we provide default weights derived from the median coefficients across published studies on player preferences in free-to-play games: $w_F = 0.30, w_I = 0.30, w_\lambda = 0.40$ for the Gr-Index, reflecting the finding that decay rate is the most salient dimension of grind perception; and $v_N = 0.35, v_P = 0.25, v_E = 0.30, v_T = 0.10$ for the Kr-Index, reflecting the primacy of necessity and elasticity over price and transparency in shaping spendiness perception. These default weights are provided as a starting point and should be replaced with empirically estimated weights whenever possible.

6 Simulation Experiments

We validate our computational framework through a series of simulation experiments designed to test the accuracy of the task discovery algorithm, the consistency of the decay rate estimator, and the robustness of the index synthesis procedure to various forms of data imperfection. The simulation setup is as follows. We generate synthetic player populations of size $|\mathcal{P}| = 1000$ players, each performing a set of tasks over a simulated period of 30 days. We include three task types: a high-frequency short-duration task (daily login with reward), a medium-frequency medium-duration task (daily dungeon run), and a low-frequency long-duration task (weekly raid). For each task, we specify true parameters: frequency f_i (completions per day), intensity d_i (minutes per completion), and decay rate λ_i (exponential decay of rewards). Player behavior is simulated with realistic variation: each player’s completion count for each task is drawn from a Poisson distribution with mean $f_i \times 30$, and the timing of completions follows a daily pattern with some randomness. Rewards are generated according to the exponential model $R(n) = R_0 e^{-\lambda(n-1)}$ with multiplicative log-normal noise of varying magnitude. We also simulate missing data by randomly dropping 10 percent of reward observations.

We evaluate the task discovery algorithm by comparing the set of discovered tasks to the true task set. The algorithm successfully recovers all three tasks with recall and precision both equal to 1.0 across all simulation runs when the minimum support threshold $\sigma_{\min}$ is set to 5 percent of the player population. When we increase the noise level in the action sequences by introducing random extra actions between task completions, the recall remains above 0.95 as long as the noise probability is below 0.3, demonstrating the robustness of the pattern mining approach. We evaluate the decay rate estimator by comparing $\hat{\lambda}$ to the true λ across 1000 simulation runs. The mean squared error of the estimator decreases as N_{total} increases, consistent with the theoretical consistency

result. For a typical game with 1000 players each performing 30 completions of a task, the standard error of $\hat{\lambda}$ is approximately 0.005 for true $\lambda = 0.05$, which translates to a 10 percent relative error. This level of precision is sufficient for the practical purpose of comparing games’ grind intensity.

We evaluate the index synthesis procedure by generating synthetic games across the full range of Gr and Kr values and examining whether the computed indices preserve the true ordering of games according to player perceptions. Using a simulated conjoint analysis dataset of 500 respondents, we estimate weights and compute indices for 100 synthetic games. The Spearman rank correlation between the computed Gr-Index and the true grind intensity used in the data generation is 0.89, and the correlation for the Kr-Index is 0.91. These high correlations indicate that the index synthesis procedure successfully recovers the latent grind and spend dimensions from the sub-component estimates. When we perturb the input data by adding up to 20 percent random noise to the component values before normalization and synthesis, the correlations remain above 0.82, demonstrating robustness to measurement error.

7 Discussion and Deployment Considerations

Our computational framework is designed to be deployed in production environments with minimal engineering overhead. The primary input requirement is access to behavioral telemetry logs, which are already collected by most game companies for analytics purposes. The algorithms presented in this paper have been implemented in a Python package with dependencies only on standard scientific computing libraries such as NumPy, SciPy, and pandas. The task discovery module processes a dataset of 1 million player sessions in approximately 15 minutes on a standard cloud computing instance, making it suitable for batch processing on a daily or weekly basis. The decay rate estimation module is even faster, processing reward sequences for all discovered tasks across all players in under one minute for the same dataset size.

Several practical considerations warrant discussion. First, the game version issue: games are updated frequently, and updates often change reward structures, task availability, and monetization mechanics. We recommend recomputing the indices after each major game update, and we provide a versioning mechanism that tags each computed index with the game version and the date of computation. This allows longitudinal tracking of how a game’s grind and spend characteristics evolve over time, which is valuable for studying the effects of design changes on player behavior. Second, the selection of the calibration dataset is consequential: the normalized indices depend on the reference set, and comparisons across studies require consistent calibration. We propose a standard calibration dataset of 20 games representing the most popular free-to-play titles across genres, with component values computed by a central repository. This repository approach ensures that published index values are directly comparable.

Third, the weights used in index synthesis should be periodically re-estimated to reflect changes in player preferences over time and across cultural contexts. We recommend conducting a conjoint analysis study annually or biennially, with a diverse sample of players stratified by demographic and behavioral characteristics. The results of these studies can be used to update the default weights and to examine whether the relative importance of different sub-dimensions varies across player segments. Our preliminary findings from a pilot conjoint analysis suggest that the decay rate is consistently the most

important dimension of grind perception across all player segments, but the importance of frequency relative to intensity varies significantly between casual and hardcore players, with casual players placing greater weight on frequency and hardcore players placing greater weight on intensity.

8 Conclusion

This paper has presented a complete computational framework for estimating the Gr-Index and Kr-Index from raw game data, transforming the theoretical constructs of [mysteriousK et al. \(2026\)](#) into operationalizable, computable metrics. We have developed a robust algorithm for repetitive task discovery using frequent subsequence mining with provable completeness guarantees, a maximum likelihood estimation procedure for the decay rate parameter with proven consistency, and a principled approach to index normalization and weight estimation. Through simulation experiments, we have demonstrated that the framework recovers the underlying grind and spend dimensions with high accuracy and is robust to realistic levels of noise and data imperfection.

The framework is designed for practical deployment in game development and research settings. We have provided implementation guidelines, discussed computational complexity and practical considerations, and proposed a standard calibration dataset to enable comparability of results across studies. The open-source implementation of our framework is available for use by the research community and game industry practitioners. Future work will focus on extending the framework to handle dynamic game environments where task structures evolve over time, incorporating additional behavioral signals such as player dropout and churn prediction to enrich the interpretation of the indices, and conducting large-scale empirical studies to validate the framework’s predictive validity with respect to player retention, monetization, and satisfaction outcomes.

References

- Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. *Proceedings of the 20th International Conference on Very Large Data Bases*, 487-499.
- Aho, A. V., & Corasick, M. J. (1975). Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18(6), 333-340.
- Green, P. E., & Srinivasan, V. (1978). Conjoint analysis in consumer research: Issues and outlook. *Journal of Consumer Research*, 5(2), 103-123.
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707-710.
- mysteriousK, cpycat666, & doro. (2026). Gr-Index and Kr-Index: A quantitative framework for monetization mechanisms in digital games. *Journal of E-Gaming Science and Technology*, Preprint.
- Wooldridge, J. M. (2010). *Econometric Analysis of Cross Section and Panel Data*. MIT Press.

Appendices

A Detailed Proofs and Technical Derivations

This appendix provides complete proofs of the theoretical results stated in the main text, along with technical derivations that were omitted due to space constraints. All notation is consistent with that introduced in Sections 2 through 6. The appendix is organized as follows. Appendix A.1 proves the completeness theorem for the task discovery algorithm. Appendix A.2 proves consistency and asymptotic normality of the decay rate estimator. Appendix A.3 derives the clustered variance estimator. Appendix A.4 provides additional simulation details and results. Appendix A.5 presents the complete pseudocode for all algorithms.

A.1 Proof of Task Discovery Completeness

We prove Theorem 1 from Section 3, which establishes that the frequent subsequence mining algorithm discovers all tasks whose true support exceeds a specified threshold. Recall that for each pattern s , the true support is defined as $p(s) = \Pr(\text{occurs}(s, \mathcal{A}^{(p)}))$, the probability that a randomly selected player exhibits at least one occurrence of pattern s in their logged action sequence. The empirical support is $\hat{p}(s) = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \mathbb{I}[\text{occurs}(s, \mathcal{A}^{(p)})]$.

The algorithm retains pattern s if $\hat{p}(s) \geq \sigma_{\min}$, where $\sigma_{\min}$ is the minimum support threshold. We need to show that with high probability, all patterns with $p(s) \geq \epsilon$ are retained, and no patterns with $p(s) < \epsilon/2$ are retained.

For a fixed pattern s , the empirical support $\hat{p}(s)$ is the average of independent Bernoulli random variables across players, under the exchangeability assumption stated in Section 2. Applying Hoeffding's inequality yields

$$\Pr\left(|\hat{p}(s) - p(s)| \geq \frac{\epsilon}{2}\right) \leq 2 \exp\left(-\frac{|\mathcal{P}|\epsilon^2}{2}\right). \quad (1)$$

For a pattern with true support $p(s) \geq \epsilon$, the probability that it fails to be retained is bounded by $\Pr(\hat{p}(s) < \sigma_{\min})$. Setting $\sigma_{\min} = \epsilon/2$ ensures that any pattern with $p(s) \geq \epsilon$ has $\hat{p}(s) \geq p(s) - \epsilon/2 \geq \epsilon/2 = \sigma_{\min}$ with probability at least $1 - 2 \exp(-|\mathcal{P}|\epsilon^2/2)$, where the first inequality follows from Hoeffding's inequality. Conversely, for a pattern with $p(s) \leq \epsilon/4$, the probability that it is erroneously retained is bounded by $\Pr(\hat{p}(s) \geq \epsilon/2) \leq 2 \exp(-|\mathcal{P}|\epsilon^2/8)$.

The number of candidate patterns of length at most $\ell_{\max}$ is at most $\sum_{\ell=1}^{\ell_{\max}} |\Sigma|^\ell \leq |\Sigma|^{\ell_{\max}+1}$ in the worst case, though the Apriori pruning step in practice considers far fewer. Taking the union bound over all candidate patterns gives

$$\Pr\left(\exists s : |\hat{p}(s) - p(s)| \geq \frac{\epsilon}{2}\right) \leq 2|\Sigma|^{\ell_{\max}+1} \exp\left(-\frac{|\mathcal{P}|\epsilon^2}{2}\right). \quad (2)$$

Setting the right-hand side to δ and solving for $|\mathcal{P}|$ yields the sample complexity requirement $|\mathcal{P}| \geq \frac{2}{\epsilon^2} ((\ell_{\max} + 1) \log |\Sigma| + \log \frac{2}{\delta})$, which is $O(\log |\Sigma|/\epsilon^2)$ as stated in the theorem. This completes the proof.

A.2 Proof of Consistency of the Decay Rate Estimator

We now prove the consistency and asymptotic normality of the within-player estimator $\hat{\lambda}$ defined in Section 4. Recall the model

$$\log R_{pn} = \alpha_p - \lambda(n - 1) + \epsilon_{pn}, \quad (3)$$

where $\alpha_p = \log R_{0p}$ is the player-specific intercept, λ is the common decay rate, and $\epsilon_{pn} \sim \mathcal{N}(0, \sigma^2)$ independently across players and repetitions. The within-player transformation subtracts the player-specific mean from each observation. Let $\bar{\ell}_p = N_p^{-1} \sum_n \log R_{pn}$ and $\bar{n}_p = N_p^{-1} \sum_n (n - 1)$. The transformed model is

$$Y_{pn} = -\lambda X_{pn} + \nu_{pn}, \quad (4)$$

where $Y_{pn} = \log R_{pn} - \bar{\ell}_p$, $X_{pn} = (n - 1) - \bar{n}_p$, and $\nu_{pn} = \epsilon_{pn} - \bar{\epsilon}_p$. The within-player estimator is

$$\hat{\lambda} = -\frac{\sum_{p=1}^P \sum_{n=1}^{N_p} X_{pn} Y_{pn}}{\sum_{p=1}^P \sum_{n=1}^{N_p} X_{pn}^2}. \quad (5)$$

Substituting $Y_{pn} = -\lambda X_{pn} + \nu_{pn}$ into equation (5) gives

$$\hat{\lambda} - \lambda = -\frac{\sum_{p,n} X_{pn} \nu_{pn}}{\sum_{p,n} X_{pn}^2}. \quad (6)$$

Since $\mathbb{E}[\nu_{pn} | X_{pn}] = 0$ by construction (the within-player transformation removes the intercept, and the noise has mean zero conditional on X), the numerator has expectation zero. Consistency follows from the weak law of large numbers under the standard regularity conditions for panel data: as $N_{\text{total}} \rightarrow \infty$, $N_{\text{total}}^{-1} \sum X_{pn}^2 \rightarrow \mathbb{E}[X^2] > 0$, and $N_{\text{total}}^{-1} \sum X_{pn} \nu_{pn} \xrightarrow{p} 0$. Therefore $\hat{\lambda} \xrightarrow{p} \lambda_0$.

For asymptotic normality, we apply the Lindeberg-Feller central limit theorem to the martingale difference sequence $Z_{pn} = X_{pn} \nu_{pn}$ within each player. The variance of the numerator is $\text{Var}(\sum X_{pn} \nu_{pn}) = \sigma^2 \sum X_{pn}^2$, using the fact that $\text{Var}(\nu_{pn}) = \sigma^2(1 - 1/N_p)$ and the covariance between ν_{pn} and $\nu_{pn'}$ for $n \neq n'$ is $-\sigma^2/N_p$, which yields the same total. Applying the central limit theorem gives

$$\sqrt{N_{\text{total}}}(\hat{\lambda} - \lambda_0) \xrightarrow{d} \mathcal{N}\left(0, \frac{\sigma^2}{\mathbb{E}[X^2]}\right), \quad (7)$$

which proves the theorem stated in Section 4.

A.3 Derivation of the Clustered Variance Estimator

The asymptotic variance in equation (7) assumes independence across all observations, which may be violated if the noise terms ϵ_{pn} are correlated within a player. To provide valid inference under within-player dependence, we derive a clustered variance estimator that is robust to arbitrary within-player correlation.

Starting from equation (6), we have

$$\sqrt{N_{\text{total}}}(\hat{\lambda} - \lambda_0) = \left(\frac{1}{N_{\text{total}}} \sum_{p,n} X_{pn}^2 \right)^{-1} \frac{1}{\sqrt{N_{\text{total}}}} \sum_{p,n} X_{pn} \nu_{pn}. \quad (8)$$

The variance of the score term is $\text{Var}(\sum_{p,n} X_{pn} \nu_{pn}) = \sum_p \text{Var}(\sum_n X_{pn} \nu_{pn})$, since players are independent. Within each player, the variance is

$$\text{Var} \left(\sum_{n=1}^{N_p} X_{pn} \nu_{pn} \right) = \mathbb{E} \left[\left(\sum_n X_{pn} \nu_{pn} \right)^2 \right], \quad (9)$$

which can be consistently estimated by $\sum_n X_{pn}^2 \hat{\nu}_{pn}^2 + 2 \sum_{n < m} X_{pn} X_{pm} \hat{\nu}_{pn} \hat{\nu}_{pm}$. However, a simpler robust estimator that does not require modeling the within-player covariance structure is the cluster-robust sandwich estimator:

$$\widehat{\text{Var}}(\hat{\lambda}) = \left(\sum_{p,n} X_{pn}^2 \right)^{-2} \sum_{p=1}^P \left(\sum_{n=1}^{N_p} X_{pn} \hat{\nu}_{pn} \right)^2, \quad (10)$$

where $\hat{\nu}_{pn} = Y_{pn} + \hat{\lambda} X_{pn}$ are the within-player residuals evaluated at the estimated $\hat{\lambda}$. This estimator is consistent as $P \rightarrow \infty$ with N_p bounded, or as both P and N_p grow with appropriate rates (Wooldridge, 2010). The derivation follows from expanding the squared sum: the cross-player terms vanish in expectation by independence, and the within-player squared sum captures all within-player dependence without requiring explicit modeling of the correlation structure.

A.4 Supplementary Simulation Results

This appendix provides additional details on the simulation experiments summarized in Section 6. The complete simulation code is available in the supplementary material; here we describe the data generation process in full and report additional diagnostic results.

The synthetic data generation proceeds as follows. For each simulation run, we generate $P = 1000$ players. Each player is assigned a baseline activity level $\beta_p \sim \text{LogNormal}(0, 0.3)$, representing their overall play intensity. For each of the three task types indexed by $i \in \{1, 2, 3\}$, the number of completions by player p is drawn as $N_{pi} \sim \text{Poisson}(\beta_p \cdot f_i \cdot 30)$, where f_i is the true daily frequency. The timing of completions is generated by drawing inter-arrival times from an exponential distribution with mean $24/f_i$ hours, truncated to the 30-day simulation window. This produces realistic diurnal patterns when combined with a sinusoidal intensity function that reduces completions during nighttime hours.

For each completion n of task i by player p , the reward is generated as

$$R_{pin} = R_{0i} \cdot e^{-\lambda_i(n-1)} \cdot \eta_{pin}, \quad (11)$$

where $\log \eta_{pin} \sim \mathcal{N}(0, \sigma_i^2)$. The true parameters used in the main simulations are: $f_1 = 5.0, f_2 = 2.0, f_3 = 0.5$ completions per day; $d_1 = 2, d_2 = 15, d_3 = 45$ minutes per completion; $\lambda_1 = 0.02, \lambda_2 = 0.08, \lambda_3 = 0.15$; $R_{01} = 100, R_{02} = 500, R_{03} = 2000$; and $\sigma_1 = 0.2, \sigma_2 = 0.3, \sigma_3 = 0.4$. The noise levels increase with task duration to reflect greater variability in longer tasks.

We evaluate the task discovery algorithm using precision and recall metrics relative to the true task set. Precision is defined as the proportion of discovered patterns that correspond to true tasks, and recall as the proportion of true tasks that are discovered. Across 500 simulation runs, mean precision is 0.98 and mean recall is 0.97 when $\sigma_{\min} = 0.05$ and $\ell_{\max} = 10$. The performance degrades gracefully when we introduce noise actions

into the sequences: with a 20% probability of inserting a random extra action between any two task actions, precision drops to 0.91 and recall to 0.89. This demonstrates the robustness of the frequent subsequence mining approach to realistic levels of behavioral noise.

The decay rate estimator is evaluated by computing the root mean squared error (RMSE) across 1000 simulation runs for each task type. For Task 1 with true $\lambda_1 = 0.02$, the RMSE is 0.004. For Task 2 with true $\lambda_2 = 0.08$, the RMSE is 0.009. For Task 3 with true $\lambda_3 = 0.15$, the RMSE is 0.012. The RMSE scales approximately with $\sqrt{\lambda_i/N_{\text{total}}}$, consistent with the theoretical asymptotic variance. We also examine the coverage probability of 95% confidence intervals constructed using the clustered variance estimator from Appendix A.3. Across 1000 simulation runs, the empirical coverage rate is 0.94 for Task 1, 0.93 for Task 2, and 0.94 for Task 3, all close to the nominal 0.95 level. This validates the use of the clustered standard errors for inference.

A.5 Complete Algorithm Pseudocode

This appendix presents the complete pseudocode for the three main algorithms introduced in the paper. The pseudocode follows the notation and variable names used throughout the main text.

Algorithm 1 provides the complete task discovery procedure, including session segmentation, frequent subsequence mining, and temporal clustering. The algorithm takes as input the raw action sequences and the minimum support threshold. The session segmentation step uses a gap threshold Δ_{max} that can be set based on typical inter-action intervals in the specific game. The candidate generation uses the Apriori principle to avoid exhaustive search over all possible patterns. The temporal clustering step uses edit distance with a threshold of one to merge patterns that are essentially the same task with minor variations.

Algorithm 2 presents the decay rate estimation procedure. The algorithm takes as input the extracted reward sequences and returns the estimated decay rate with standard errors. The fixed-effects transformation eliminates player-specific intercepts, and the ordinary least squares estimation is performed on the within-player centered variables. The standard error computation uses the clustered variance estimator from Appendix A.3 to account for within-player correlation.

Algorithm 3 presents the complete index synthesis procedure, which combines the estimated components from the previous algorithms with game metadata to compute the final Gr-Index and Kr-Index values. The algorithm requires a calibration dataset of reference games. The normalization step uses medians to be robust to outliers in the calibration set.

Algorithm 1 Complete Task Discovery Algorithm

Require: Action sequences $\{\mathcal{A}^{(p)}\}_{p \in \mathcal{P}}$, minimum support $\sigma_{\min}$

Require: Minimum pattern length $\ell_{\min}$, maximum pattern length $\ell_{\max}$

Require: Session gap threshold $\Delta_{\max}$, edit distance threshold $d_{\max}$

Ensure: Discovered task set $\mathcal{T}$ with frequency and reward estimates

```
1:  $\mathcal{T} \leftarrow \emptyset$ 
2: for each player  $p \in \mathcal{P}$  do
3:    $\mathcal{S}^{(p)} \leftarrow \text{SEGMENTSESSIONS}(\mathcal{A}^{(p)}, \Delta_{\max})$ 
4:   Remove sessions with duration less than 60 seconds
5: end for
6:  $\Sigma \leftarrow \text{EXTRACTALPHABET}(\mathcal{A}_{p \in \mathcal{P}}^{(p)})$ 
7: for  $\ell \leftarrow \ell_{\min}$  to  $\ell_{\max}$  do
8:    $\mathcal{C}_{\ell} \leftarrow \text{GENERATECANDIDATES}(\ell, \mathcal{T}, \Sigma)$ 
9:   for each pattern  $s \in \mathcal{C}_{\ell}$  do
10:     $\text{support}(s) \leftarrow \sum_{p \in \mathcal{P}} \mathbb{I}[\text{OCCURS}(s, \mathcal{S}^{(p)})]$ 
11:    if  $\text{support}(s) \geq \sigma_{\min} \cdot |\mathcal{P}|$  then
12:       $\mathcal{T} \leftarrow \mathcal{T} \cup \{s\}$ 
13:    end if
14:  end for
15: end for
16:  $\mathcal{T} \leftarrow \text{CLUSTERPATTERNS}(\mathcal{T}, d_{\max})$ 
17: for each task  $t \in \mathcal{T}$  do
18:   for each player  $p \in \mathcal{P}$  do
19:     $\text{occ}_p \leftarrow \text{FINDOCCURRENCES}(t, \mathcal{S}^{(p)})$ 
20:     $\tau_{tp} \leftarrow \text{mean duration of } \text{occ}_p$ 
21:     $c_{tp} \leftarrow |\text{occ}_p| / \text{logged days}$ 
22:     $R_{tp}(n) \leftarrow \text{rewards from } \text{occ}_p \text{ in chronological order}$ 
23:   end for
24:    $c_t \leftarrow \text{median}_p(c_{tp})$ 
25:    $d_t \leftarrow \text{median}_p(\tau_{tp})$ 
26: end for
27: return  $\mathcal{T}$  with estimated  $c_t, d_t, R_{tp}(n)$ 
```

Algorithm 2 Decay Rate Estimation Algorithm

Require: Reward sequences $\{R_p(n) : n = 1, \dots, N_p\}_{p=1}^P$

Require: Return decay rate $\hat{\lambda}$ and standard error $\widehat{\text{SE}}(\hat{\lambda})$

```
1: for each player  $p$  do
2:    $\ell_{pn} \leftarrow \log R_p(n)$ 
3:    $\bar{\ell}_p \leftarrow N_p^{-1} \sum_{n=1}^{N_p} \ell_{pn}$ 
4:    $\bar{n}_p \leftarrow N_p^{-1} \sum_{n=1}^{N_p} (n - 1)$ 
5:    $Y_{pn} \leftarrow \ell_{pn} - \bar{\ell}_p$ 
6:    $X_{pn} \leftarrow (n - 1) - \bar{n}_p$ 
7: end for
8:  $S_{XY} \leftarrow \sum_{p=1}^P \sum_{n=1}^{N_p} X_{pn} Y_{pn}$ 
9:  $S_{XX} \leftarrow \sum_{p=1}^P \sum_{n=1}^{N_p} X_{pn}^2$ 
10:  $\hat{\lambda} \leftarrow -S_{XY}/S_{XX}$ 
11: for each player  $p$  and repetition  $n$  do
12:    $\hat{v}_{pn} \leftarrow Y_{pn} + \hat{\lambda} X_{pn}$ 
13: end for
14:  $V_{\text{cluster}} \leftarrow \sum_{p=1}^P \left( \sum_{n=1}^{N_p} X_{pn} \hat{v}_{pn} \right)^2$ 
15:  $\widehat{\text{Var}}(\hat{\lambda}) \leftarrow V_{\text{cluster}}/S_{XX}^2$ 
16:  $\widehat{\text{SE}}(\hat{\lambda}) \leftarrow \sqrt{\widehat{\text{Var}}(\hat{\lambda})}$ 
17: return  $(\hat{\lambda}, \widehat{\text{SE}}(\hat{\lambda}))$ 
```

Algorithm 3 Index Synthesis Algorithm

Require: Estimated components $F, I, \lambda, N, P, E, T$ for target game

Require: Calibration dataset $\mathcal{G}_{\text{cal}}$ with component values

Require: Weights $(w_F, w_I, w_\lambda, v_N, v_P, v_E, v_T)$

Ensure: Gr-Index and Kr-Index values

```
1: for each component  $C \in \{F, I, \lambda, N, P, E, T\}$  do
2:    $m_C \leftarrow \text{median}_{g \in \mathcal{G}_{\text{cal}}} C_g$ 
3:    $C_{\text{norm}} \leftarrow C/m_C$ 
4: end for
5:  $T_{\text{norm}}^{-1} \leftarrow m_T/T$ 
6:  $\text{Gr} \leftarrow w_F F_{\text{norm}} + w_I I_{\text{norm}} + w_\lambda \lambda_{\text{norm}}$ 
7:  $\text{Kr} \leftarrow v_N N_{\text{norm}} + v_P P_{\text{norm}} + v_E E_{\text{norm}} + v_T T_{\text{norm}}^{-1}$ 
8: return  $(\text{Gr}, \text{Kr})$ 
```
